R Comprehensive Cheat Sheet

High-yield syntax, grammar, workflows, plotting, data wrangling, and time-series code patterns.
Built from your course slides plus current official R / tidyverse references.

Base R | Data frames | dplyr | ggplot2 | Time series

What belongs in your muscle memory

- Assignment (`<-`), indexing `[ ]`/`[[ ]]`/`$`, sequences `:`, help `?` and `help()`
- Vectors, factors, matrices, lists, data frames, tibbles, missing values `NA`
- Import data, inspect structure, clean columns, summarise, plot, save results
- For this module: `ts()`, `SMA()`, `HoltWinters()`, `acf()`, `pacf()`, `fft()`, `forecast()`, `ssa()`

Read this as a fast reference. Syntax first, then examples, then course-specific time-series code.

1. Core R grammar

R is vectorised. Many operations work on whole vectors, not single values. A lot of speed and elegance comes from thinking in whole columns / whole series.

Pattern	Meaning
`x <- 5`	assign
`# comment`	comment
`x[3]`	3rd element
`x[2:5]`	slice
`x[x > 0]`	logical filter
`mydf$col`	column by name
`mylist[[1]]`	extract one item
`seq(0,1,by=.1)`	sequence
`mean(x, na.rm=TRUE)`	ignore missing values

Bare minimum syntax

```
# comments
x <- 5
y <- c(1, 3, 5, 7)
z <- x + 10

# comparison and logic
y > 3
y[y > 3]
(1 < 2) & (3 < 4)
(1 < 2) | (5 < 4)
!TRUE
```

Help + environment

- `getwd()` current working directory
- `setwd("path")` change folder
- `ls()` list objects
- `rm(x)` remove object, `rm(list = ls())` clear workspace
- `?mean`, `help("lm")`, `example(plot)`

The main beginner distinction is: atomic vector (one type), list (can mix types), matrix (2D one type), data frame (table, columns can differ), and factor (categorical data).

```
num <- c(2, 4, 6)
chr <- c("a", "b")
logi <- c(TRUE, FALSE, TRUE)
fac <- factor(c("low", "high", "low"))
mat <- matrix(1:6, nrow = 2, byrow = TRUE)
df  <- data.frame(id = 1:3, score = c(10, 12, 11))
lst <- list(name = "Zahra", marks = c(18, 19))
```

Check	Useful calls
type / class	`typeof(x)`, `class(x)`, `str(x)`
size	`length(x)`, `nrow(df)`, `ncol(df)`, `dim(mat)`
names	`names(df)`, `colnames(df)`, `rownames(df)`
convert	`as.numeric()`, `as.character()`, `as.factor()`

```
# factor control
grade <- factor(c("B", "A", "C"), levels = c("A", "B", "C"), ordered = TRUE)
levels(grade)

# missing values
x <- c(1, NA, 3, NaN)
is.na(x)
mean(x, na.rm = TRUE)
```

Indexing is where many R bugs begin. Learn the difference between `[ ]`, `[[ ]]`, and $.

Object	Syntax	Meaning
vector	`x[2]`	2nd element, returns vector
vector	`x[c(1,4)]`	multiple positions
vector	`x[x > 0]`	logical indexing
list	`lst[1]`	sub-list
list	`lst[[1]]`	single element itself
data frame	`df[1:5, ]`	rows 1 to 5, all columns
data frame	`df[, c("a","b")]`	chosen columns

data frame	df$score	column access

Slicing and filtering

```
x <- c(10, 20, 30, 40)
x[-1]          # drop first
x[c(TRUE, FALSE, TRUE, FALSE)]

df <- data.frame(a = 1:4, b = c(9, 8, 7, 6))
df[1:2, ]      # first two rows
df[, "b"]     # one column
df[df$b >= 8, ]
```

Common indexing traps

- `df[1]` returns a data frame column; `df[[1]]` returns the raw column vector
- `x[0]` is empty; negative indices drop positions
- Character indices need exact names more safely than partial matching
- For lists, use `[[ ]]` when you need the stored object itself

4. Import, inspect, clean, export

Base import/export

```
# base R
mydata <- read.csv("data/myfile.csv")
head(mydata)
names(mydata)
summary(mydata)

# write out
write.csv(mydata, "clean_data.csv", row.names = FALSE)
```

Tidyverse import/export

```
# readr (friendly + fast)
# install.packages("readr")
library(readr)
df <- read_csv("data/myfile.csv")
write_csv(df, "clean_data.csv")
```

Inspect fast	Meaning
`head(df)`	first rows
`tail(df)`	last rows
`str(df)`	compact structure
`summary(df)`	quick descriptive summary
`unique(df$col)`	distinct values
`table(df$col)`	frequency table

Quick column edits

```
# rename / create / drop with base R
names(df)[names(df) == "old"] <- "new"
df$ratio <- df$x / df$y
df$dropme <- NULL
```

5. Descriptive statistics and maths

Task	Base R
sum / mean / median	`sum(x)`, `mean(x)`, `median(x)`
spread	`sd(x)`, `var(x)`, `range(x)`, `IQR(x)`
quantiles	`quantile(x)`
count	`length(x)`, `length(unique(x))`
sorting	`sort(x)`, `order(x)`, `rank(x)`
math	`log(x)`, `exp(x)`, `sqrt(x)`, `abs(x)`

Matches the style from the beginner screenshot + common summary calls

```
summary(mpg)
sd(mpg)
length(mpg)

cor(x, y, use = "complete.obs")
quantile(x, probs = c(.25, .5, .75), na.rm = TRUE)
```

6. Control flow + functions

Control flow

```
if (mean(x) > 0) {
  print("positive average")
} else {
  print("not positive")
}

for (i in 1:5) {
  print(i^2)
}

while (x < 100) {
  x <- x + 10
}
```

Writing a function

```
scale01 <- function(x) {
  (x - min(x, na.rm = TRUE)) /
    (max(x, na.rm = TRUE) - min(x, na.rm = TRUE))
}

scale01(c(2, 4, 8))
```

Vectorise when you can

- Prefer `x[x > 0]`, `ifelse()`, `pmin()`, `pmax()`, `apply()` / `lapply()` before slow manual loops
- Loops are still fine when the logic is naturally sequential, such as recursive forecasts

7. dplyr workflow

The high-frequency verbs are `select()`, `filter()`, `mutate()`, `arrange()`, `group_by()`, `summarise()`. They are built for readable data-wrangling pipelines.

Canonical pipeline

```
library(dplyr)

df %>%
  filter(score >= 50) %>%
  mutate(ratio = x / y) %>%
  group_by(group) %>%
  summarise(mean_score = mean(score, na.rm = TRUE),
            n = n()) %>%
  arrange(desc(mean_score))
```

Verb	What it does
`select()`	keep / rename columns
`filter()`	keep rows that meet a condition
`mutate()`	create or transform columns
`arrange()`	sort rows
`group_by()`	declare grouping
`summarise()`	collapse groups to summary rows
`count()`	quick counts
`across()`	do same operation on many columns

Selectors and grouped summaries

```
# useful selectors
df %>% select(name, starts_with("temp"), where(is.numeric))

# grouped summaries
df %>%
  group_by(month) %>%
  summarise(across(c(sales, cost), mean, na.rm = TRUE))
```

8. ggplot2

Every plot starts with `ggplot(data, aes(...))`, then you add layers with `+`.

Minimal line plot

```
library(ggplot2)

ggplot(df, aes(x = time, y = value)) +
  geom_line(linewidth = 0.7) +
  labs(title = "Time series", x = "Time", y = "Value") +
  theme_minimal()
```

Three very common plot patterns

```
ggplot(df, aes(x = group, y = score, fill = group)) +
  geom_boxplot()

ggplot(df, aes(x = score)) +
  geom_histogram(bins = 30)

ggplot(df, aes(x = x, y = y, colour = class)) +
  geom_point() +
  geom_smooth(se = FALSE, method = "lm")
```

Aesthetic mappings

- `aes(x, y, colour, fill, shape, size, linetype)` maps variables to visual properties
- Set a constant outside `aes()`: `geom_line(colour = "steelblue")`
- Use `ggsave("plot.png", width=7, height=4, dpi=300)` to save a plot

9. Base graphics still matter in lectures

Base plotting grammar from lecture style

```
plot(x, type = "l", main = "Series")
points(x, pch = 19, cex = 0.5)
hist(x)
boxplot(x)
barplot(tab)
par(mfrow = c(2, 1))
```

Exactly the lecture pattern used repeatedly in the R supplement

```
plot(t, x, type = "l", main = "signal x")
points(t, x, pch = 19, cex = 0.4)
```

10. Time-series essentials in R

Base ts object

```
# create a monthly time series
x_ts <- ts(x, start = c(2024, 1), frequency = 12)
plot(x_ts)
frequency(x_ts)
start(x_ts); end(x_ts)
```

Decomposition + moving average

```
# decomposition and smoothing
decomp <- decompose(x_ts)
plot(decomp)

# moving average (lecture 2 uses TTR)
library(TTR)
ySMA <- SMA(x_ts, n = 10)
```

Lecture 2 smoothing + forecast pattern

```
# exponential smoothing
fit_hw <- HoltWinters(x_ts, seasonal = "additive")
plot(fit_hw)

# forecast next 12 periods
library(forecast)
plot(forecast(fit_hw, h = 12))
```

Diagnostics seen later in the module

```
# serial dependence diagnostics
acf(x_ts)
pacf(x_ts)

# differencing
y <- diff(log(x_ts))
plot(y, type = "l")
```

Lecture 5 R supplement: spectral analysis

```
# periodogram function from the R supplement
Pgram <- function(xts, title = "time series") {
  n <- length(xts)
  I <- abs(fft(xts - mean(xts)))^2 / n
  P <- (4 / n) * I[1:(n / 2)]
  nu <- 0:(n / 2 - 1) / n
  plot(nu, P, type = "l", xlab = "Frequency",
       ylab = "Scaled Periodogram", main = title)
  cbind(nu, P)
}
```

Fast interpretation map

- `ts()` gives R a time index structure
- `SMA()` smooths by averaging a moving window
- `HoltWinters()` fits exponential smoothing models
- `acf()` and `pacf()` are for dependence diagnostics
- `fft()` / periodogram code moves you into frequency-domain thinking

11. Course-specific code patterns from the slides

Lecture 2

```
# Lecture 2: logistic / moving average style
library(TTR)
ySMA <- SMA(x, n = 10)
plot(x, type = "l", col = "gray", main = "SMA on Logistic Trend")
```

Lecture 2

```
# Lecture 2: milk example
library(fma)
library(TTR)
smoothMA <- SMA(milkTS, m)
plot(milkTS, type = "l")
```

Lecture 2

```
# Lecture 2: exponential smoothing and forecasting
smoothES <- HoltWinters(milkTS, alpha, beta = FALSE, gamma = FALS
plot(smoothES)
HWmodel <- HoltWinters(milkTS, seasonal = "additive")
library(forecast)
plot(forecast(HWmodel, h = 12))
```

Lecture 5 R supplement

```
# Lecture 5 supplement: filtering a signal
install.packages("signal")
library(signal)
install.packages("audio")
library(audio)

fCoef <- fir1(fLen, fPassInterval, "pass")
y2 <- filter(fCoef, x2)
plot(t, y2, type = "l", main = "filtered signal")
```

Lecture 14

```
# Lecture 14: airpass workflow
plot(airpass[,3], type = "l", main = "airpass data")
plot(log(airpass[,3]), type = "l", main = "log: airpass data")
y <- diff(log(airpass[,3]))
acf(y)
pacf(y)
```

Lecture 15

```
# Lecture 15: SSA forecasting
install.packages("Rssa")
library(Rssa)
airpassTS <- ts(airpass[,3], start = c(airpass[1,2], 1), frequenc
s <- ssa(airpassTS, L = 24)
f <- forecast(s, groups = list(1:6), method = "bootstrap-recurren
plot(f, shadecols = "green", type = "l")
```

12. Model fitting quick-start

Regression basics

```
# linear model
fit <- lm(y ~ x1 + x2, data = df)
summary(fit)
coef(fit)
predict(fit, newdata = new_df)

# generalized workflow
plot(fit)
```

Common forecasting workflow

```
# classical ARIMA path (outside your early lectures, but common i
# install.packages("forecast")
library(forecast)
fit <- auto.arima(x_ts)
fc  <- forecast(fit, h = 12)
plot(fc)
```

13. Strings, dates, and useful cleaning

Strings with base R

```
tolower(x)
toupper(x)
gsub(" ", "_", names(df))
substr(x, 1, 3)
paste("mean =", round(mean(x), 2))
```

Dates + duplicate removal

```
# dates
dates <- as.Date(c("2026-04-21", "2026-04-22"))
format(dates, "%Y-%m")

# remove duplicates
unique(df)
df[!duplicated(df), ]
```

14. Cheat formulas and one-line recipes

Need this?	Use this
top rows	`head(df, 10)`
random sample	`sample(x, 5)`
sort descending	`sort(x, decreasing = TRUE)`
replace missing	`x[is.na(x)] <- 0`
subset rows	`subset(df, score > 50)`
drop missing rows	`na.omit(df)`
cross-tab	`table(df$a, df$b)`
save workspace object	`saveRDS(obj, "obj.rds")`
read it back	`readRDS("obj.rds")`

15. Beginner error decoder

Message / symptom	Likely cause
object not found	name typo or object not created
could not find function	package not loaded with `library()`
subscript out of bounds	index too large / wrong dimension
NAs introduced by coercion	bad type conversion, often text -> number
replacement has x rows, data has y	column length mismatch
plot looks weird	wrong type, unsorted x-axis, or grouping issue

Golden habits

- Run `str(df)` before analysis
- Check missing values before summaries and models
- Name objects clearly and keep a clean script in sections
- Save plots and outputs with explicit file names
- For time series, always know the sampling frequency and what one observation means

16. References used for this cheat sheet

- Course PDFs: ma3508_lecture2.pdf (moving averages and Holt-Winters examples), ma3508_lecture5_R_supplement.pdf (signal, filter, periodogram code), ma3508_lecture14.pdf (airpass, acf/pacf, fft usage), ma3508_lecture15.pdf (Rssa / forecast examples).
- Official R manuals: An Introduction to R; R Language Definition; R Data Import/Export; utils::read.table and utils::write.table docs.
- Official or package references: ggplot2 reference index and aes(); dplyr reference for summarise(), select(), arrange(); forecast package docs; Rssa package docs.